

Ordenação de Grandes Volumes de Dados Genômicos usando MPI - Entrega até 28/09

Trabalho 1 - Programação Paralela

September 8, 2025

1 Descrição

Muitos problemas em ciência de dados e computação científica exigem a manipulação de grandes volumes de informação. A ordenação de dados é um passo fundamental em várias aplicações, como análise estatística, bancos de dados, processamento de logs e simulação científica. Neste projeto, vamos desenvolver uma aplicação paralela para ordenar grandes volumes de dados genômicos (sequências de DNA) para facilitar análises comparativas e busca de padrões. A ordenação é crucial para operações eficientes como busca binária e identificação de sequências similares. **Este trabalho será realizado por grupos de até duas pessoas**, onde cada grupo deverá implementar sua solução paralela utilizando a biblioteca MPI (Message Passing Interface) e a linguagem C para ordenar grandes volumes de dados em um ambiente com memória distribuída.

2 Objetivos

1. Implementar uma das soluções paralelas vista em sala usando o MPI para ordenação de um conjunto de sequências genômicas.
2. Comparar o desempenho com uma solução sequencial.
3. Realizar experimentos em um computador com pelo menos 8 processadores. Sua solução deve ter desempenho melhor que o sequencial. A nota estará relacionada ao desempenho.
4. Elaborar um relatório descrevendo a solução, a descrição da máquina onde os testes foram realizados, os experimentos dos resultados obtidos bem como uma análise dos resultados.

3 Descrição da Aplicação

- A entrada será um arquivo de texto contendo uma lista de fragmentos de DNA (um por linha).
- O programa deverá distribuir os dados entre os processos MPI, realizar a ordenação de forma cooperativa e, ao final, reunir o resultado ordenado em um único processo mestre, que escreverá o resultado em um arquivo de saída.

4 Exemplos de Entrada

```
ATGCTAGCT
GCTAGCTAG
ATCGATCG
GATCGATC
```

5 Exemplos de Saída

```
ATCGATCG
ATGCTAGCT
GATCGATC
GCTAGCTAG
```

6 Solução Sequencial

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_SEQ_LENGTH 100
#define DNA_CHARS "ACGT"

void generate_dna_sequence(char* seq, int length) {
    for (int i = 0; i < length; i++) {
        seq[i] = DNA_CHARS[rand() % 4];
    }
    seq[length] = '\0';
}

int compare_dna(const void* a, const void* b) {
    return strcmp((char*)a, (char*)b);
}

void sequential_sort(char** data, int n) {
    qsort(data, n, sizeof(char*), compare_dna);
}
```

7 Descrição dos Experimentos

1. Gere diferentes conjuntos de dados de entrada com:
 - 100 mil sequências
 - 1 milhão de sequências
 - 10 milhões de sequências
2. Execute o programa em diferentes números de processadores (2, 4, 8, ...) e meça o tempo de execução.
3. Compare os resultados com a versão sequencial da ordenação implementado em C/C++.
4. Produza gráficos mostrando:
 - Tempo de execução vs. tamanho do conjunto de dados.
 - Tempo de execução vs. número de processadores.
 - Speedup ($T_{seq.}(n)/T_{par}(n)$)

8 O que entregar

1. Código fonte da solução paralela (em C/C++ com MPI).
2. Relatório técnico contendo:
 - Descrição da estratégia adotada (como os dados foram distribuídos, algoritmo de ordenação paralela escolhido).
 - Resultados experimentais com tabelas e gráficos.
 - Análise comparativa entre execução sequencial e paralela.
 - Discussão sobre ganhos de desempenho e limitações observadas.
3. **Cada membro do grupo deve gerar e entregar um único arquivo .tgz** com o comando dado a seguir e depois submeter este arquivo no AVA:

```
tar czvf felca_pereira.tgz SortPar.c relatorio.pdf
```

onde `tar` é um utilitário de arquivamento do Linux, `felca_pereira` é o seu login e `.tgz` é uma extensão que identifica este arquivo.

Bom Trabalho!